

ConfigManager: Persistent Configuration for the Robot

Every robot needs a place to live its settings — speed limits, directory paths, launch templates, and runtime flags. `ConfigManager` is that place. It is the first dependency of almost every other module in the stack, which is why it sits at the top of the reading order.

The Factory Pattern for Safe Initialization

Python constructors cannot return errors, so a direct `__init__` that opens a file is hard to test and hard to reason about. `ConfigManager` solves this with a class-method factory:

```
@classmethod
def create(cls, config_file: str) → "ConfigManager":
    manager = cls(config_file)
    manager.load_config()
    manager.ensure_subdirs()
    manager.detect_test_environment()
    return manager
```

`create` sequences the steps that must happen before the object is usable: load the YAML, create missing directories, and detect if we are running under `pytest`. Callers always use `create`; the constructor is intentionally bare — just stores paths and initializes variables to an empty dict.

Variables as a Typed Dictionary

All configuration lives in a single flat dict `self.variables`, loaded from YAML. The smart part is `set_variable`, which coerces string input to the right Python type before storing:

```
def set_variable(self, name: str, value: object):
    if not isinstance(value, str):
        self.variables[name] = value
    elif value.lower() in ["true", "false"]:
        self.variables[name] = value.lower() == "true"
    elif self.is_number(value):
        self.variables[name] = self.convert_number(value)
    else:
        self.variables[name] = value
    self.save_config()
```

The priority order is: non-string passes through, then bool detection, then numeric detection, then raw string. This matters because YAML already types most values — the coercion path is only exercised when values arrive as strings from the CLI.

```
def convert_number(self, value: str) → int | float:
    if "." in value:
        return float(value)
    return int(value)
```

The dot-presence heuristic for int vs float is simple and correct for robot config values.

Path Resolution

The robot stores maps, logs, and other artifacts relative to the config directory. `resolve_path` handles the ambiguity between absolute and relative paths:

```
def resolve_path(self, path_str: str) → Path:
    path = Path(path_str).expanduser()
    if path.is_absolute():
        return path
    return (self.control_dir / path).resolve()
```

`ensure_subdirs` calls this to create the `maps/` and `logs/` directories on first use, so callers never have to check.

Test Environment Detection

A subtle but important design: `detect_test_environment` sets `dry_run = True` when `pytest` or `unittest` is present in `sys.modules`. This propagates to `MovementApi.cmd_vel_helper`, which skips real hardware commands in tests without requiring explicit mocking of the hardware layer.

```
def detect_test_environment(self):
    if "pytest" in sys.modules or "unittest" in sys.modules:
        self.variables["dry_run"] = True
```

Launch Templates

The config file also carries launch templates — named command templates with default parameters. `ConfigManager` exposes these as a plain dict; `ProcessApi` consumes them to build `LaunchConfig` objects. The separation keeps `ConfigManager` free of any subprocess knowledge.

```
def get_launch_templates(self) → dict:
    return self.variables.get("launch_templates", {})
```

Observations

- **No schema validation.** Missing required keys (e.g., `linear_speed`) surface as `None` from `get_variable`, caught only at point of use. A startup `validate()` would catch config errors earlier.

- **Save on every set.** `set_variable` calls `save_config()` unconditionally. A transactional API or explicit `flush()` would be more efficient for bulk updates.
-

Flat namespace. All variables are top-level. Nested sections (`hardware`, `nav`, `speech`) would make the config file more readable but require deeper access helpers.

version: “1.0” generated: “2026-05-04” —

BaseApi

`base_api.py` defines the abstract base class shared by all ROS2 API nodes in this package. It is the point where ROS2’s `Node` machinery meets the package’s configuration and logging conventions.

Multiple Inheritance

```
class BaseApi(Node, ABC):
    def __init__(self, node_name: str, config_manager: cm.ConfigManager = None):
        super().__init__(node_name)
        self.config = config_manager or cm.ConfigManager()
```

`BaseApi` inherits from both `Node` (ROS2) and `ABC` (Python abstract base class). The `ABC` inheritance isn’t strictly necessary here — there are no `@abstractmethod` declarations — but it signals that `BaseApi` is not meant to be instantiated directly.

The `config_manager` parameter defaults to a freshly constructed `ConfigManager()` if none is provided. This is a convenience for ad-hoc use, but in production all API nodes receive the same `ConfigManager` instance from `RobotController`, which is important for shared state (e.g., `dry_run` flag, speed variables).

Logging Conventions

Four logging methods wrap ROS2’s logger with a twist:

```
def log_debug(self, message: str):
    self.get_logger().debug(message)

def log_info(self, message: str):
    print(message)          # ← goes to stdout, not ROS2 log

def log_warn(self, message: str):
    self.get_logger().warn(message)
```

```
def log_error(self, message: str):
    self.get_logger().error(message)
```

`log_info` routes to `print()` rather than the ROS2 logger. This is intentional: the CLI user sees stdout directly in their terminal; ROS2 info-level log output typically goes to a separate log sink and isn't visible interactively. `debug`, `warn`, and `error` use the ROS2 logger so they appear in `ros2 topic echo /rosout` and log files.

Bounds Checking

```
def check_bounds(self, value: float, min_val: float, max_val: float, name: str) → bool:
    if not (min_val ≤ value ≤ max_val):
        self.log_warn(f"{name} out of bounds: {value}. Must be [{min_val}, {max_val}]")
        return False
    return True
```

This helper is used by `MovementApi.check_velocity_limits` before every velocity command. Keeping it in `BaseApi` makes it available to any future API class that also needs range validation.

Node Lifecycle

`destroy_node` overrides the ROS2 default to log a shutdown message before calling `super().destroy_node()`. This gives a consistent teardown trace in logs when the CLI exits.

Observations

- The fallback `ConfigManager()` in `__init__` calls `ConfigManager(config_file)` with no argument, which will raise `TypeError`. The default is never exercised in practice (all callsites pass a config), but it's misleading as written.
-

ABC is imported but not used for its primary purpose. Future API classes that must implement specific methods could use `@abstractmethod` here.

version: "1.1" generated: "2026-05-04" —

MovementApi: Velocity Control and Odometry

`MovementApi` is the thin ROS2 layer between the robot's wheels and the command layer. It publishes `Twist` messages to `/cmd_vel` and subscribes to `/odom`.

All movement commands in the stack eventually call one of its three primitives: `cmd_vel_helper`, `turn_amount`, or `stop`.

Inheriting from BaseApi

`MovementApi` extends `BaseApi`, which wraps `rclpy.Node`. This gives it access to `create_publisher`, `create_subscription`, `log_info`, and the `config` property without re-implementing node boilerplate.

```
class MovementApi(base.BaseApi):
    def __init__(self, config_manager: cm.ConfigManager = None):
        super().__init__("movement_api", config_manager)
        self.cmd_vel_pub = self.create_publisher(Twist, "/cmd_vel", 1)
        self.odom_sub = self.create_subscription(
            Odometry, "/odom", self.odom_callback, 10
        )
        self._validate_config()
        self.current_pose = None
```

A queue depth of 1 for `cmd_vel` is intentional: if the robot can't keep up with commands, the newest command should win, not queue behind stale ones.

Speed as Config Properties

Rather than capturing speeds at construction time, each speed is a live property reading from `ConfigManager`. This means `config set linear_speed 0.4` takes effect on the next movement call without restarting the node:

```
@property
def linear(self) → float:
    return self.config.get_variable("linear_speed")
```

The same pattern applies to `angular`, `linear_min`, `linear_max`, `angular_min`, `angular_max`.

Config Validation at Startup

The six speed variables are required. `_validate_config` checks all of them at construction time so failures are loud and early rather than silent `None` math errors mid-movement:

```
def _validate_config(self):
    required_vars = ["linear_speed", "angular_speed",
                    "linear_min", "linear_max", "angular_min", "angular_max"]
    missing = [v for v in required_vars if self.config.get_variable(v) is None]
    if missing:
        raise ValueError(f"Missing required configuration variables: {' '.join(missing)}")
```

The Central Primitive: cmd_vel_helper

All movement boils down to this method — publish a Twist at a fixed rate for a given duration, then stop:

```
def cmd_vel_helper(self, linear: float, angular: float, seconds: float):
    if not self.blocking_ok:
        raise RuntimeError("cmd_vel_helper called from a non-blocking context")
    if not self.check_velocity_limits(linear, angular):
        return
    if self.config.is_dry_run():
        self.log_info(f"DRY RUN: Would move linear={linear}, angular={angular} for {seconds}s")
        return

    twist = Twist()
    twist.linear.x = linear
    twist.angular.z = angular
    start_time = time.time()
    rate_hz = 10
    sleep_duration = 1.0 / rate_hz
    while rclpy.ok() and (time.time() - start_time) < seconds:
        self.cmd_vel_pub.publish(twist)
        rclpy.spin_once(self, timeout_sec=sleep_duration)
        time.sleep(sleep_duration)

    twist.linear.x = 0.0
    twist.angular.z = 0.0
    self.cmd_vel_pub.publish(twist)
```

The `blocking_ok` flag guards against calling this from a lifecycle node's spin loop, where blocking would deadlock. The explicit zero-velocity stop at the end is a safety measure: the robot stops even if the caller crashes after calling this.

Movement Primitives

The higher-level methods are all thin wrappers over `cmd_vel_helper`:

```
def move_dist(self, distance: float):
    seconds = abs(distance) / self.linear
    actual_speed = self.linear if distance >= 0 else -self.linear
    self.cmd_vel_helper(actual_speed, 0.0, seconds)

def turn_amount(self, angle: float):
    seconds = abs(angle) / self.angular
    angular_speed = self.angular if angle >= 0 else -self.angular
    self.cmd_vel_helper(0.0, angular_speed, seconds)
```

```
def turn_degrees(self, degrees: float):
    self.turn_amount(math.radians(degrees))
```

Sign conventions: positive distance = forward, positive angle = counterclockwise (ROS standard).

Odometry Subscription

The subscription stores the latest pose but does not act on it. This is a foundation for future closed-loop control:

```
def odom_callback(self, msg):
    self.current_pose = msg.pose.pose
```

Observations

- **Open-loop movement.** `move_dist` computes time from speed; it does not use odometry feedback. Wheel slip or hardware variation accumulates as error.
- **10 Hz fixed rate.** The publish loop runs at a hardcoded 10 Hz. A configurable rate or ROS timer would be more idiomatic.
-

Blocking architecture. `cmd_vel_helper` blocks the calling thread. This is simple but prevents concurrent operations and is incompatible with lifecycle node contexts.

version: “1.0” generated: “2026-05-04” —

IntentApi

`intent_api.py` is the outbound side of the semantic behavior pipeline. It publishes structured intent messages to the `/intent` topic, which the `behavior_manager_node` consumes.

Design

```
class IntentApi(base.BaseApi):
    def __init__(self, config_manager: cm.ConfigManager = None):
        super().__init__("intent_api", config_manager)
        self.intent_pub = self.create_publisher(String, "/intent", 10)

    def publish(self, name: str, source: str, slots: dict) → None:
        msg = String()
        msg.data = json.dumps({"name": name, "source": source, "slots": slots})
```

```

self.intent_pub.publish(msg)
self.log_info(f"Intent published: {msg.data}")

```

The intent is a JSON-serialised dict over `std_msgs/String`. The three fields match the `Intent` dataclass in `behavior_manager.py`:

Field	Purpose
<code>name</code>	Action name, e.g. <code>"describe_scene"</code> , <code>"stop"</code>
<code>source</code>	Who published it — <code>"cli"</code> for CLI commands, <code>"voice"</code> for voice input
<code>slots</code>	Typed parameters, e.g. <code>{"object_type": "chair"}</code>

Why `std_msgs/String` and not a Custom Message

Using JSON over `String` means the intent topic requires no custom message type at compile time. Any ROS2 node — including Python scripts, shell tools, and the CLI — can publish or echo `/intent` without depending on this package's message definitions. The tradeoff is no compile-time type checking on the payload.

Publisher Lifecycle in `RobotController`

`RobotController` lazily constructs `IntentApi` on first use:

```

@property
def intent(self) → IntentApi:
    if self._intent is None:
        self._intent = IntentApi(self.config)
        time.sleep(0.5)  # allow DDS discovery
    return self._intent

```

The `time.sleep(0.5)` after construction gives DDS time to discover the subscriber before the first publish. Without this delay, the first intent message may be dropped if the `behavior_manager_node` subscriber hasn't yet registered with the local DDS participant.

Observations

- The `source` field is hardcoded to `"cli"` in all `RobotController` publish methods. If multiple interfaces (REST, TUI) are added later, passing the source as a parameter would be more correct.
-

There is no acknowledgement mechanism. The CLI receives a `CommandResponse(True, ...)` immediately after publishing, regardless of whether the behavior manager received or handled the intent.

version: “1.0” generated: “2026-05-04” —

CalibrationApi

`calibration_api.py` provides scripted movement patterns for calibration and stress testing. It sits above `MovementApi` in the dependency chain and delegates all actual motion to it.

Structure

```
class CalibrationApi(base.BaseApi):
    def __init__(self, movement_api: movement.MovementApi, config_manager: cm.ConfigManager):
        super().__init__("calibration_api", config_manager)
        self.movement = movement_api
```

`CalibrationApi` receives an existing `MovementApi` instance rather than constructing its own. This matters because `MovementApi` is a ROS2 node (it registers publishers and subscribers). Creating a second `MovementApi` would create a second `movement_api` node with duplicate pub/sub connections.

Square Pattern

```
def run_square_pattern(self, side_length: float):
    for i in range(4):
        self.movement.move_dist(side_length)
        self.movement.turn_amount(math.pi / 2)
```

Drives four equal sides with 90-degree turns between them. Because `move_dist` and `turn_amount` are open-loop (time-based), accumulated heading error means the robot does not return to exactly the starting point. The square pattern is useful for measuring that accumulated error.

Rotation Stress Test

```
def run_rotate_stress(self):
    slow_angular = self.config.get_variable("stress_test_rotation_speed") or 0.2
    full_rotation = 2 * math.pi
    ...
    while rclpy.ok():
        for rotation in range(10):
            time_for_rotation = full_rotation / slow_angular
            self.movement.cmd_vel_helper(0.0, slow_angular, time_for_rotation)
```

Runs in an infinite loop until `KeyboardInterrupt`. Each cycle completes 10 full rotations. The rotation speed comes from config (`stress_test_rotation_speed`), allowing tuning without code changes. `_print_elapsed_time` logs progress every 10 seconds so the operator knows the test is still running.

Circle Stress Test

```
def run_circle_stress(self, diameter: float):
    radius = diameter / 2.0
    circumference = 2 * math.pi * radius
    circle_time = circumference / linear_speed
    self.movement.cmd_vel_helper(linear_speed, angular_speed, circle_time)
```

For a circle of a given diameter, the required angular velocity is v / r (linear speed divided by radius). This value must be set in config as `angular_speed` before running — there’s no automatic calculation of the correct angular speed from the diameter. If the configured `angular_speed` doesn’t match `linear_speed / radius`, the robot traces an arc rather than a circle.

Observations

- The circle stress test silently produces wrong geometry if `angular_speed` isn’t set to `linear_speed / radius`. A `log_warn` when the computed ratio differs significantly from the configured value would help.
- Both stress tests catch `KeyboardInterrupt` and call `self.movement.stop()`. This is good practice — without it, a `Ctrl+C` mid-motion would leave the robot moving.
-

`_print_elapsed_time` is a helper that returns `current_time` if it logged, `last_print_time` if not. The pattern is slightly awkward; a simple counter or `time.time()` modulo check would be cleaner.

version: “1.1” generated: “2026-05-04” —

ProcessApi: Subprocess Lifecycle Management

`ProcessApi` owns everything that involves spawning, monitoring, and killing OS-level processes. It has two distinct modes of operation: **async launch** (fire-and-forget background processes) and **sync execution** (blocking commands that return output). Both are used by `RobotController` for different purposes.

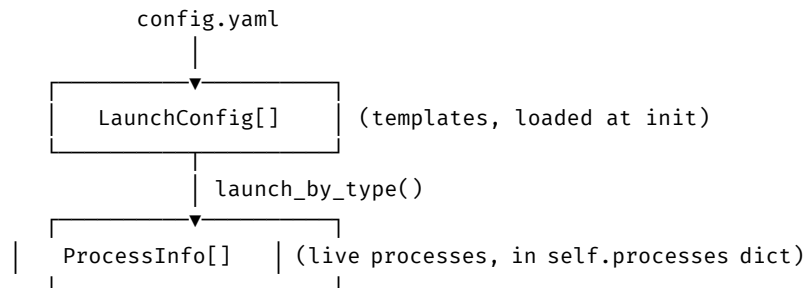
Data Structures

Two dataclasses carry all process state:

```
@dataclass
class ProcessInfo:
    process: subprocess.Popen
    command: str
    output: List[str]
    is_running: bool
    pid: int
    start_time: float
    log_file: Optional[str] = None
```

```
@dataclass
class LaunchConfig:
    launch_type: str
    command_template: str
    description: str
    default_params: Dict[str, str]
```

ProcessInfo is the runtime state of a running process. LaunchConfig is the static template loaded from YAML config.



Loading Launch Templates

ProcessApi.__init__ calls _load_launch_configs to convert the raw YAML dict into typed LaunchConfig objects:

```
def _load_launch_configs(self):
    templates = self.config.get_launch_templates() if self.config else {}
    self.launch_configs = {}
    for launch_type, template in templates.items():
        self.launch_configs[launch_type] = LaunchConfig(
            launch_type=launch_type,
            command_template=template.get("command", ""),
            description=template.get("description", ""),
```

```

        default_params=template.get("default_params", {})
    )

```

Async Launch: `launch_command`

The async path launches a process in a new session (`start_new_session=True`) so it survives terminal disconnection. A background thread captures all stdout to a log file and to the console:

```

proc = subprocess.Popen(
    command, shell=True,
    stdout=subprocess.PIPE, stderr=subprocess.STDOUT,
    stdin=subprocess.DEVNULL,
    text=True, bufsize=1,
    start_new_session=True,
)

```

`start_new_session=True` creates a new process group, which matters for `kill_ros_process`: if the PID is the group leader, `SIGTERM` is sent to `-pgid` (the whole group), not just the parent.

A UUID is returned as the process handle. `RobotController` stores this UUID in `launch_process_ids` keyed by launch type.

Sync Execution: `run_command_sync`

Map saving and serialization need to run a command and wait for its result. `CommandConfig` bundles command, log name, and timeout:

```

@dataclass
class CommandConfig:
    command: str
    log_name: Optional[str]
    timeout: float

def run_command_sync(self, config: CommandConfig) → tuple[bool, str, Optional[str]]:
    result = subprocess.run(
        config.command, shell=True,
        capture_output=True, text=True,
        timeout=config.timeout
    )
    return (result.returncode == 0, result.stdout + result.stderr, log_file_path)

```

Returns (success, output, `log_file_path`) so callers can report both the outcome and where to find details.

Killing Processes

`kill_ros_process` handles the case where the target may be a process group leader — in which case `kill -TERM -PID` sends `SIGTERM` to the entire group:

```
target = f"-{pid}" if is_group_leader else str(pid)
subprocess.run(["kill", "-TERM", target], check=True, timeout=2)
```

If `SIGTERM` fails, it escalates to `SIGKILL`. This graceful-then-force pattern avoids leaving zombie processes.

Process Status Queries

`is_process_running` uses `poll()` to detect silent process death without blocking:

```
def is_process_running(self, process_id: str) → bool:
    proc_info = self.processes[process_id]
    proc_info.process.poll()
    if proc_info.process.returncode is not None:
        proc_info.is_running = False
        return False
    return True
```

`get_process_pid` provides the OS PID for external display:

```
def get_process_pid(self, process_id: str) → int | str:
    info = self.processes.get(process_id)
    return info.pid if info else "unknown"
```

Listing ROS and Launch Processes

`list_ros_processes` and `list_launch_processes` shell out to `ps aux` and `ps axo` respectively, filtering by ROS-related keywords. These are display-only utilities — they scan all OS processes, not just managed ones:

```
ps aux → filter ros/nav2/slam/rviz keywords → tabular output
ps axo pid,pgid,ppid → filter ros2 launch → parent-only display
```

Observations

- **Parameter formatting is complex.** `launch_by_type` has distinct code paths for `bl` commands, `ros2 run`, and launch files. This fragility could be replaced with a strategy object per command type.
- **`list_ros_processes` leaks shell knowledge.** The hardcoded keyword list `["ros2", "nav2", "slam", "rviz", ...]` will miss new components and is not tested.
-

Circular import via string. `kill_ros_process` and `list_*` import `CommandResponse` inline to avoid circular imports. The fix is to move `CommandResponse` to its own module.

version: "1.0" generated: "2026-05-04" —

Announcement Contract

`announcement_contract.py` defines the shared data model for messages published to the `/announcement` topic. It is the contract between producers (behavior manager, voice input) and consumers (speech output node).

Why a Separate Contract Module

Multiple nodes in different files need to agree on the `/announcement` message structure. A shared module avoids each node reimplementing its own serialization logic and prevents the producer and consumer from drifting apart.

Priority Constants

The module establishes an ordered urgency scale:

```
PRIORITY_SAFETY = 0          # highest – obstacle, emergency
PRIORITY_STATUS = 1         # battery low, localization lost
PRIORITY_QUERY_REPLY = 2    # answer to a user question
PRIORITY_ACTION_ACK = 3     # "turning left"
PRIORITY_DISCOVERY = 4      # "I see a chair"
PRIORITY_CHITCHAT = 5       # lowest – greetings, idle commentary
```

Lower numbers = higher priority. The scale is not yet enforced by a priority queue in the speech output node, but is the intended semantic for future queueing logic.

Dual Import Strategy

The module uses `try/except ImportError` in two places to allow use outside a ROS2 environment:

```
try:
    from builtin_interfaces.msg import Time
except ImportError:
    @dataclass
    class Time:
        sec: int = 0
        nanosec: int = 0
```

And similarly for `AnnouncementMsg`. This lets unit tests import the module without a ROS2 installation. The stub classes provide just enough structure for pure-Python assertions.

The Announcement Dataclass

```
@dataclass(frozen=True)
class Announcement:
    text: str
    priority: int = PRIORITY_CHITCHAT
    stamp: Time | None = None
    source: str = "unknown"
    dedup_key: str = ""
```

`frozen=True` makes instances hashable and immutable — safe to pass across ROS2 callbacks.

`dedup_key` is a forward-looking field intended for suppressing repeated identical announcements (e.g., a “battery low” warning that would otherwise fire every few seconds).

Serialization Paths

Producer		Consumer
Announcement		
→ <code>to_payload()</code>	→ JSON string ↔	<code>from_payload()</code> → Announcement
→ <code>to_msg()</code>	→ AnnouncementMsg ↔	<code>from_msg()</code> → Announcement

`to_payload` / `from_payload` use a plain JSON string over `std_msgs/String`. This allows nodes that don’t have the custom message type compiled to still participate via the JSON path.

`from_payload` is defensive: if the payload is not valid JSON, the whole string is treated as the announcement text. This lets legacy publishers still work.

Factory Helpers

```
def make_announcement_payload(text, *, priority, source, dedup_key) → str
def make_announcement_msg(text, *, priority, source, dedup_key, stamp) → AnnouncementMsg
```

Convenience wrappers so call sites don’t need to construct `Announcement` and immediately call `.to_payload()` or `.to_msg()`.

Observations

- The `dedup_key` field is defined but nothing currently populates it with meaningful values. The speech output node does not suppress duplicates.

•

Priority is not acted on anywhere. A future priority queue in `SpeechOutputNode` would need to buffer incoming announcements and sort by priority before speaking.

version: "1.0" generated: "2026-05-04" —

Movement Commands

`movement_commands.py` defines the command registry entries for all robot motion commands. It is the translation layer between user-facing CLI vocabulary and `RobotController` method names.

What This Module Is and Isn't

This module contains *no logic*. It returns a dict of `CommandDef` objects. All motion implementation is in `RobotController` and `MovementApi`. The purpose of this file is to declare:

- What command names the CLI recognises
- What parameters each command expects
- Which `RobotController` method to call

The Registry

```
def build_movement_commands() → dict[str, cd.CommandDef]:
    return {
        "move.distance": CommandDef("move_distance", [ParameterDef("distance", float, True, ...)]),
        "move.time":      CommandDef("move_for_time", [ParameterDef("seconds", float, True, ...)]),
        "turn.time":      CommandDef("turn_for_time", [ParameterDef("seconds", float, True, ...)]),
        "turn.radians":   CommandDef("turn_radians", [ParameterDef("radians", float, True, ...)]),
        "turn.degrees":   CommandDef("turn_degrees", [ParameterDef("degrees", float, True, ...)]),
        "move.forward":   CommandDef("move_forward", [ParameterDef("meters", float, True, ...)]),
        "move.backward":  CommandDef("move_backward", [ParameterDef("meters", float, True, ...)]),
        "turn.clockwise": CommandDef("turn_clockwise", [ParameterDef("degrees", float, True, ...)]),
        "turn.counterclockwise": CommandDef("turn_counterclockwise", [ParameterDef("degrees", float, True, ...)]),
    }
```

Command Name Convention

Keys use `group.subcommand` dot notation. `CommandDispatcher` treats the dot as a separator between the command group and the subcommand variant. `SimpleParser` maps user input "move forward 1.5" to the key "move.forward" by joining the first two resolved tokens with a dot.

Semantic Duplication

Notice that `move.distance` / `move.forward` / `move.backward` all ultimately call `move_dist` on `MovementApi`, with `move.forward` forcing positive and `move.backward` forcing negative distance. This redundancy is intentional: it lets users speak in whichever vocabulary feels natural (`move distance -1.0` vs `move backward 1.0`).

Observations

- All parameters are `required=True` with `default=None`. There are no optional motion parameters. Adding defaults (e.g., a default distance of 1.0 metre) would make one-word commands like `move forward` work.

•

`turn.radians` and `turn.degrees` both exist, which is good for users who think in either unit. They call different `RobotController` methods which in turn call different `MovementApi` methods (`turn_amount` for radians, `turn_degrees` which converts and calls `turn_amount`).

version: “1.0” generated: “2026-05-04” —

Launch Commands

`launch_commands.py` defines the command registry entries for managing ROS2 launch processes. These commands expose `ProcessApi`’s lifecycle management through the CLI.

The Registry

```
"launch.list": CommandDef("launch_list", [], ... )
"launch.info": CommandDef("launch_info", [ParameterDef("launch_type", str, True, ...)], ... )
"launch.start": CommandDef("launch_start", [
    ParameterDef("launch_type", str, True, None, "Launch type (nav, slam, map)"),
    ParameterDef("use_sim_time", bool, False, None, "Use simulation time"),
    ParameterDef("map", str, False, None, "Map filename"),
    ParameterDef("map_name", str, False, None, "Map name for map launch type"),
], ... )
"launch.stop": CommandDef("launch_stop", [ParameterDef("launch_type", str, True, ...)], ... )
```

Parameter Design for `launch.start`

`launch.start` has four parameters but only `launch_type` is required. The optional parameters (`use_sim_time`, `map`, `map_name`) are passed as `**kwargs` through

`RobotController.launch_start` into `ProcessApi.launch_by_type`, which adds non-None values to the command template.

This means the CLI user writes:

```
launch start nav --map basement
```

and `ProcessApi` resolves `basement` to the full YAML path under `maps_dir` and appends `map:=/full/path/basement.yaml` to the `ros2 launch` command.

What `launch.list` Shows

`launch.list` calls `RobotController.launch_list`, which queries `ProcessApi.get_available_launch_types()` and formats a table:

NAME	DESCRIPTION

nav	Nav2 navigation stack
slam	SLAM Toolbox online async
map	Map server

These names come directly from the `launch_templates` keys in `config.yaml`. If the config has no templates, this command returns “No launch templates available.”

Observations

- `launch.info` returns the raw command template, which exposes implementation details (full `ros2 launch` command strings). This is useful for debugging but might be surprising to end users.
- There is no `launch.status` command to see which launch types are currently running. `robot status` includes this information but it’s buried in a larger status dump.
-

`map_name` and `map` are distinct parameters that behave differently in `ProcessApi.launch_by_type` — `map` is a map file for `nav`, while `map_name` is for the map server launch type. This is a confusing API that could be unified.

version: “1.0” generated: “2026-05-04” —

System Commands

`system_commands.py` defines the command registry entries for configuration management, scripted movement patterns, and system-level process inspection. It

bundles four conceptually distinct groups: `config`, `script`, `system`, and implicitly exposes map operations via the navigation group.

Groups Covered

`config`

<code>config.set</code>	<code>name value</code>	→ <code>RobotController.set_variable</code>
<code>config.get</code>	<code>name</code>	→ <code>RobotController.get_variable</code>
<code>config.list</code>		→ <code>RobotController.get_all_variables</code>

Exposes `ConfigManager` through the CLI. `config.list` filters out `launch_templates` from the output (it's too verbose for a terminal display).

`script`

<code>script.list</code>		→ lists available scripts
<code>script.square</code>	<code>meters</code>	→ <code>CalibrationApi.run_square_pattern</code>
<code>script.rotate_stress</code>		→ <code>CalibrationApi.run_rotate_stress</code>
<code>script.circle_stress</code>	<code>diameter</code>	→ <code>CalibrationApi.run_circle_stress</code>

These are the only commands that directly invoke multi-step, long-running motion sequences. They block the CLI until the pattern completes or the user interrupts with `Ctrl+C`.

`system`

<code>system.topics</code>	→ list active /topics via <code>MovementApi.get_topic_names_and_types</code>
<code>system.ps</code>	→ list all ROS-related OS processes (<code>ps aux</code> filtered)
<code>system.kill pid</code>	→ kill process by PID (group kill if <code>PID == PGID</code>)
<code>system.launches</code>	→ list <code>ros2</code> launch parent processes (filtered from <code>ps</code>)

`system.kill` is noteworthy: `RobotController.kill_ros_process` first checks whether the target PID is a process group leader by comparing `PID == PGID`. If it is, it sends `SIGTERM` to the whole process group (`kill -TERM -PID`). This is the correct way to shut down a `ros2` launch invocation — killing just the launcher process leaves all its children orphaned.

Observations

- `system.topics` uses `self.movement.get_topic_names_and_types()`, which creates a `MovementApi` node on first call. This has the side effect of initialising a full ROS2 subscriber set just to list topics. A lightweight dedicated node or `ros2 topic list` subprocess call would be cleaner.
- `script.rotate_stress` has no timeout parameter — it runs until `Ctrl+C`. For remote operation this is hazardous; adding an optional `--max-seconds` parameter would be safer.
-

`config.set` and `config.get` are in `system_commands.py` but the `config` group name is distinct from `system`. Future refactoring could split `config` into its own file.

version: “1.1” generated: “2026-05-04” —

RobotController: Orchestrating Robot Operations

`RobotController` is the business logic layer. It sits between the command interfaces (CLI, REST) and the ROS2 API nodes (`MovementApi`, `ProcessApi`, `CalibrationApi`, `IntentApi`). Its job is to coordinate those APIs and return `CommandResponse` objects that callers can format however they like.

Uniform Return Type

Every public method returns a `CommandResponse`:

```
@dataclass
class CommandResponse:
    success: bool
    message: str
    data: dict | None = None
```

This is a deliberate design decision. Without it, callers need to handle `None`, exceptions, tuples, and strings. With it, the CLI can always check `result.success` and print `result.message`.

Lazy API Initialization

ROS2 nodes are expensive to construct — they register with the DDS middleware and allocate system resources. `RobotController` defers construction until first use via properties:

```
@property
def movement(self) → MovementApi:
    if self.movement_node is None:
        self.movement_node = MovementApi(self.config)
    return self.movement_node
```

The backing variables (`movement_node`, `calibration_node`, `process_node`, `intent_node`) start as `None`. When `get_robot_status` inspects them, it checks for `None` directly — a non-`None` node means it was constructed and is therefore “available”.

```
nodes_status = {
    "movement_api": "running" if self.movement_node else "not available",
```

```

        "calibration_api": "running" if self.calibration_node else "not available",
        "process_api": "running" if self.process_node else "not available",
    }

```

Launch Process Tracking

RobotController.__init__ pre-populates launch_process_ids with None values for every known launch type:

```

self.launch_process_ids = dict.fromkeys(self.config.get_launch_templates().keys())

```

This means iteration over launch_process_ids always covers all configured launch types, whether running or not — no surprises when listing status.

```

launch_process_ids = {
    "nav": "3f2a- ..." # running - stores UUID from ProcessApi
    "slam": None           # not running
    "map": None            # not running
}

```

Launch Start and Stop

launch_start checks for a conflict (already running), then delegates to ProcessApi.launch_by_type and stores the returned UUID:

```

def launch_start(self, launch_type: str, **kwargs) → CommandResponse:
    if self.is_launch_running(launch_type):
        return CommandResponse(False, f"{launch_type} is already running, stop it first")
    try:
        process_id = self.process.launch_by_type(launch_type, **kwargs)
        self.launch_process_ids[launch_type] = process_id
        return CommandResponse(True, f"Started {launch_type}", {"process_id": process_id})
    except ValueError as e:
        return CommandResponse(False, str(e))

```

launch_stop mirrors this: look up the UUID, verify it is actually running, kill it, clear the UUID.

Map Operations

Map save and serialize share a common pattern: build a shell command, wrap it in a CommandConfig, run it synchronously, and interpret the result:

```

def map_path(self):
    map_name = self.config.get_variable("map_name")
    if not map_name:
        return None, None, None
    self.config.ensure_subdirs()
    maps_dir = self.config.get_maps_dir()

```

```

    return map_name, maps_dir, maps_dir / map_name

def run_map_command(self, config: CommandConfig, success_msg: str, error_prefix: str) → CommandResponse:
    success, output, log_file = self.process.run_command_sync(config)
    if success:
        return CommandResponse(True, success_msg + (f"\nLog: {log_file}" if log_file else ""))
    ...

```

map_path is extracted because both map_save and map_serialize need the same three values: name, directory, and full path. Without the helper, that logic would be duplicated.

Intent Publishing

The publish_intent methods dispatch named intents to the behavior manager via IntentApi:

```

def publish_intent(self, name: str, slots: dict) → CommandResponse:
    self.intent.publish(name, "cli", slots)
    return CommandResponse(True, f"Intent published: {name}")

```

The intent property has a 0.5-second sleep after first construction to allow DDS publisher-subscriber discovery before the first publish. This is a known ROS2 startup race condition.

Delegating to ProcessApi

Process management methods delegate entirely to ProcessApi:

```

def list_ros_processes(self) → CommandResponse:
    return self.process.list_ros_processes()

def kill_ros_process(self, pid: int) → CommandResponse:
    return self.process.kill_ros_process(pid)

```

This keeps RobotController free of subprocess and ps parsing logic.

Observations

- **turn_degrees breaks the pattern.** It returns self.movement.turn_degrees(degrees) directly instead of a CommandResponse. MovementApi.turn_degrees returns None, so callers get None back — a latent bug.
- **No error handling on movement calls.** Methods like move_distance call self.movement.move_dist without try/except. A ROS2 exception (e.g., node not initialized) would propagate unhandled to the CLI.
-

list_maps complexity. The file-grouping logic (stem → extensions) is 20 lines that could be a helper. It also conflates map files and any other file in the directory.

version: “2.0” generated: “2026-05-06” —

CommandDispatcher: Command Registry and Execution

CommandDispatcher is the single entry point for all command execution. The CLI, REST API, and any future interface call `dispatcher.execute(name, params)` or `dispatcher.dispatch_text(text)` and get back a `CommandResponse`. The dispatcher handles parameter validation, type coercion, and method dispatch — callers never touch `RobotController` methods directly.

Registry as a Dict

The command registry is a plain `dict[str, CommandDef]` built at construction time:

```
def _build_command_registry(self) → dict[str, cd.CommandDef]:
    commands = {}
    commands.update(mov_cmd.build_movement_commands())
    commands.update(ctrl_cmd.build_control_commands())
    commands.update(nav_cmd.build_navigation_commands())
    commands.update(lch_cmd.build_launch_commands())
    commands.update(sys_cmd.build_system_commands())
    return commands
```

Each `build_*` function returns a dict of `"group.command" → CommandDef`. Intent commands and semantic commands were in this registry prior to F15/T07; they are now handled entirely by `dispatch_text` routing. The flat merge means duplicate names from different modules would silently shadow each other.

The Execute Path

```
dispatcher.execute("move.forward", {"meters": "1.5"})
|
├─ look up CommandDef in registry
├─ _validate_parameters → coerce "1.5" → 1.5 (float)
├─ getattr(robot_controller, "move_forward")
└─ method(meters=1.5) → CommandResponse
```

dispatch_text: Text-to-Command Routing

dispatch_text(text) is the entry point for free-form text input (CLI, voice transcripts):

```
def dispatch_text(self, text: str) → rc.CommandResponse:
    tokens = text.strip().split()
    command = resolve_keyword(tokens[0])
    ...
    intent_name = BEHAVIOR_COMMANDS.get(command_name)
    if intent_name is not None:
        self.publish_intent(intent_name, slots)
        return rc.CommandResponse(True, f"Intent published: {intent_name}")
    ...
    return self.execute(command_name, params)
```

Two routing paths:

- **Behavior path** — command name is in BEHAVIOR_COMMANDS → publish to /intent via IntentPublisher, return immediately.
- **Direct path** — everything else → look up CommandDef, validate params, call RobotController method.

BEHAVIOR_COMMANDS

Maps CLI command names to intent names:

```
BEHAVIOR_COMMANDS: dict[str, str] = {
    "intent.stop": "stop",
    "intent.explore": "explore",
    "intent.describe_scene": "describe_scene",
    "intent.count_objects": "count_objects",
    "scene.describe": "describe_scene",
    "scene.count": "count_objects",
}
```

The scene.* forms are the preferred interactive vocabulary; intent.* forms are kept for completeness and scripting.

Abbreviation Resolution

ABBREV_TO_FULL and FULL_NAMES are module-level constants. resolve_keyword expands short tokens:

```
ABBREV_TO_FULL = {"m": "move", "fwd": "forward", "stp": "stop", ...}
FULL_NAMES = set(ABBREV_TO_FULL.values())
```

```
def resolve_keyword(word: str) → str:
    if word in FULL_NAMES:
```



```

        return word
    return ABBREV_TO_FULL.get(word, word)

```

Unknown tokens pass through unchanged; errors surface at execution time.

Subcommand Detection

After resolving the first token, the parser checks whether the second token forms a compound command:

```

second = resolve_keyword(tokens[1])
candidate = f"{command}.{second}"
in_registry = candidate in self.commands or candidate in BEHAVIOR_COMMANDS
if second in FULL_NAMES or second != tokens[1] or in_registry:
    command_name = candidate    # compound: "move.forward"
    args = [parse_value(t) for t in tokens[2:]]
else:
    command_name = command      # simple: "stop"
    args = [parse_value(t) for t in tokens[1:]]

```

The `in_registry` check ensures scene describe routes correctly even though “describe” is not in `FULL_NAMES`.

IntentPublisher Injection

```

def __init__(self, robot_controller, intent_publisher=None):
    ...
    self.intent_publisher = intent_publisher

```

In production, leave `intent_publisher=None` — `publish_intent` creates an `IntentPublisher` lazily. In tests, pass `IntentPublisher(publish_fn=published.append)` to capture published payloads without ROS2.

Parameter Validation and Coercion

`_convert_parameter_value` handles booleans specially:

```

if param_def.param_type == bool:
    if isinstance(value, str):
        return value.lower() in ("true", "1", "yes", "on")
    return bool(value)

```

Python’s `bool("false")` is `True`, so string booleans need explicit mapping.

Observations

- **Silent shadowing.** Duplicate keys across `build_*` functions are silently overwritten.
-

Method lookup at runtime. A stale `method_name` in a `CommandDef` fails at call time, not at registry build time.

version: “2.0” generated: “2026-05-06” —

SimpleCLI

`simple_cli.py` is the top-level entry point for the robot control CLI. It assembles the full stack — config, controller, dispatcher — and provides both an interactive REPL and a non-interactive single-command mode.

Assembly at Startup

```
class SimpleCLI:
    def __init__(self):
        default_cfg = str(Path.home() / ".control" / "config.yaml")
        config_path = os.environ.get("CONTROL_CONFIG", default_cfg)
        self.config_manager = cm.ConfigManager.create(config_path)
        self.robot_controller = rc.RobotController(self.config_manager)
        self.dispatcher = cd.CommandDispatcher(self.robot_controller)
```

Construction order: `ConfigManager` → `RobotController` → `CommandDispatcher`.
`CONTROL_CONFIG` env var overrides the default config path.

`SimpleCommandParser` was removed in F15/T05. All text parsing now goes through `CommandDispatcher.dispatch_text`.

Two Modes

```
def main():
    cli = SimpleCLI()
    if len(sys.argv) > 1:
        cli.execute_command(" ".join(sys.argv[1:]))
    else:
        cli.repl()
```

Non-interactive: `ros2 run control run move forward 1.0` — executes one command and exits. Interactive REPL: `prompt_toolkit` with persistent history in `~/.control/prompt_history.txt`. Commands are also timestamped to `~/.control/command_history.txt`.

`execute_command`

```
def execute_command(self, input_text: str) → None:
    tokens = input_text.strip().split()
    first = tokens[0].lower()
```

```

if first in ("help", "hlp"):
    subcommand = resolve_keyword(tokens[1]) if len(tokens) > 1 else None
    arguments = [resolve_keyword(t) for t in tokens[2:]]
    self.handle_help(subcommand, arguments)
    return

if first in ("exit", "quit", "q", "x"):
    ...

result = self.dispatcher.dispatch_text(input_text)

```

Three paths: help handling (in-CLI), exit handling (in-CLI), everything else goes to `dispatch_text`. Help and exit are intercepted before dispatch because they require CLI-level state (`self.running`, `self.dispatcher.commands`).

Help System

```

help                → show_common_commands()
help commands       → show_all_commands()
help <group>         → show_subcommand_suggestions()
help <group> <subcommand> → load_help_file() → docs/<group>.<sub>.txt

```

`show_all_commands` dumps the full `dispatcher.commands` registry alphabetically, including behavior commands (intent/scene group). Detailed help files load from `docs/` relative to the package source root.

Error Feedback

When an unknown command is entered:

```

if "Unknown command" in result.message and "." not in tokens[0]:
    self.show_subcommand_suggestions(tokens[0], "Did you mean one of these?")

```

Lists all `<group>.*` commands if the token looks like a group name.

Observations

- `log_command` writes to file on every command with no buffering or rotation — history file grows indefinitely.
- `load_help_file` traverses three levels up from `__file__` to find `docs/`. Brittle to layout changes.
-

Empty input is silently ignored in both REPL and non-interactive mode.

version: “1.0” generated: “2026-05-06” —

IntentMapper

`intent_mapper.py` is the small translation layer between Vosk transcript text and normalized intent dictionaries. Its job is not to understand language in a general sense. It maps a constrained command vocabulary into the robot's existing intent contract, with the smallest amount of logic needed to keep the voice path predictable.

Why This Module Exists

Voice input is already constrained by the recognizer. That makes a full NLU stack unnecessary here and, more importantly, undesirable. The mapper stays as plain substring matching so it is easy to reason about, easy to test, and easy to keep aligned with the wake-word and grammar lists used by the runtime.

```
VOSK_COMMANDS = json.dumps([
    "go forward", "go backward", "turn left", "turn right",
    "what do you see", "describe the scene", "what do you see around you",
    "look around", "what's there",
    "how many", "count",
    "stop", "halt",
    "start exploring", "explore",
    "be quiet", "quiet",
    "battery", "battery level",
    "where are you", "where",
    "get status", "what's your status", "what's going on",
    "[unk]",
])
```

This list is both documentation and a recognition fence. Anything not in the list is intentionally out of scope for the voice path. The current set is trimmed to the intents the robot actually supports from voice.

Mapping Strategy

```
PHRASE_INTENTS = (
    (("stop", "halt"), "stop"),
    (("go forward",), "move_forward"),
    (("go backward",), "move_backward"),
    (("turn left",), "turn_left"),
    (("turn right",), "turn_right"),
    (("start exploring", "explore"), "start_exploring"),
    (("be quiet", "quiet"), "be_quiet"),
    (("battery",), "get_battery"),
    (("where are you", "where"), "get_location"),
    (("get status", "what's your status", "what's going on", "status"), "get_status"),
)
```

The mapper is intentionally table-driven. The data structure is the important part: it makes the accepted phrases obvious, keeps the `map_intent()` body small, and makes future additions or removals mechanical.

```
def map_intent(self, text: str) → dict | None:
    normalized_text = text.lower().strip()
    if not normalized_text or normalized_text == "[unk]":
        return None

    if contains_phrase(normalized_text, DESCRIBE_SCENE_PHRASES):
        return {"name": "describe_scene", "source": self.SOURCE, "slots": {}}

    if contains_phrase(normalized_text, COUNT_OBJECT_PHRASES):
        return make_count_objects_intent(normalized_text, self.SOURCE)

    for phrases, name in PHRASE_INTENTS:
        if contains_phrase(normalized_text, phrases):
            return {"name": name, "source": self.SOURCE, "slots": {}}

    return None
```

The function performs three steps:

1. Normalize the transcript.
2. Check for the semantic query patterns first.
3. Fall back to the command phrase table.

That order matters. A voice command like `count cans` should route into the object-count path before the generic intent matcher sees the word `count`.

Object Extraction

`count_objects` is the only intent here that carries a slot.

```
def make_count_objects_intent(text: str, source: str) → dict:
    object_type = extract_object(text)
    slots = {"object_type": object_type} if object_type else {}
    return {"name": "count_objects", "source": source, "slots": slots}
```

The extraction step is intentionally simple. It looks for known object words and fills the slot if one is present. If no object type appears in the transcript, the intent is still valid and the behavior layer can decide how to respond.

Compatibility

```
DEFAULT_MAPPER = IntentMapper()
```

```
def map_intent(text: str) → dict | None:
    return DEFAULT_MAPPER.map_intent(text)
```

The module-level wrapper keeps the old call sites working while the class-based API remains the canonical implementation.

Observations

- The mapper is deliberately conservative. If a phrase is removed from the grammar or the intent table, the voice path stops recognizing it instead of guessing.
- The current voice contract does not include `go home`, `follow me`, `sleep`, or `wake`, so those phrases are intentionally absent from this version.
- The matcher uses substring checks. That is enough for the constrained grammar and keeps the implementation easy to audit. — version: “1.0” generated: “2026-05-06” —

WakeWordDetector

`wake_word.py` provides the wake-word half of the voice loop. It keeps the openWakeWord dependency isolated behind a small class that can be configured at construction time and used from the ROS adapter or from tests with injected fakes.

Dependency Isolation

```
def _load_deps():
    try:
        import numpy as np
        import pyaudio
        from openwakeword.model import Model
    except ImportError as exc:
        raise RuntimeError(
            "Install voice deps: pip install openwakeword pyaudio numpy"
        ) from exc
    return np, pyaudio, Model
```

This lazy import pattern keeps the rest of `control` importable on machines that do not have the voice stack installed. The error only appears when the wake-word detector is actually constructed.

Model Selection

The detector now accepts a wake-word name, threshold, and wake-hit count rather than baking those values into the class. That makes it match the tuned voice parameters exposed in `control.voice.runtime`.

```
def _resolve_model_path(name_or_path: str) → str:
    if os.path.isabs(name_or_path) and os.path.exists(name_or_path):
```

```

        return name_or_path

import openwakeword as oww

models_dir = os.path.join(os.path.dirname(oww.__file__), "resources", "models")
...

```

The resolver still supports either an explicit ONNX path or a short model name. That keeps the detector useful for both the stock bundled model and custom tuned models.

Wake Detection

```

def wait_for_wake(self, ok_fn: Callable[[], bool] = lambda: True) → bool:
    self._model.reset()
    wake_hits = 0
    while ok_fn():
        audio = self._np.frombuffer(
            self._stream.read(self.FRAME_LENGTH, exception_on_overflow=False),
            dtype=self._np.int16,
        )
        score = list(self._model.predict(audio).values())[0]
    ...

```

The detector requires a configured number of consecutive wake hits before it fires. That matches the tuning workflow, where one threshold is not always enough to eliminate chatter.

The `ok_fn` callback remains the shutdown hook. In production it is `rcipy.ok`; in tests it can be a constant lambda.

Stream Lifecycle

The stream is opened and closed explicitly with `start()` and `stop()`. That is important because the wake-word loop is continuous, while the command capture phase is separate and should own the microphone during its turn.

Observations

- The detector still flushes a few frames after reopening the stream to avoid stale buffered audio.
- The code path is intentionally thin. All tuning now lives in the shared voice runtime configuration instead of this module. — version: “1.0” generated: “2026-05-06” —

SpeechTranscriber

stt.py owns the speech-to-text half of the voice loop. It turns command audio into text using Vosk, with the grammar and model path selected by the voice runtime configuration.

Vocabulary and Model Path

The transcriber no longer assumes only one hardcoded grammar. Instead it accepts the grammar from the caller and turns it into the JSON format Vosk expects.

```
def _grammar_json(grammar: list[str] | tuple[str, ...] | str | None) → str:
    if grammar is None:
        return VOSK_COMMANDS
    if isinstance(grammar, str):
        phrases = [item.strip() for item in grammar.split(",") if item.strip()]
    else:
        phrases = [str(item).strip() for item in grammar if str(item).strip()]
    if "[unk]" not in phrases:
        phrases.append("[unk]")
    return json.dumps(phrases)
```

That makes the recognizer compatible with the tuned command set in `control.voice.runtime` while still allowing the older vocabulary constant to serve as the fallback.

The model path is also configurable through the runtime's `vosk_model` value, with short keys like `small` expanded through the local path map.

Recording Loop

```
def transcribe(self) → str:
    self._rec.Reset()
    stream = self._pa.open(
        rate=16000, channels=1, format=self._pyaudio.paInt16,
        input=True, frames_per_buffer=self.CHUNK,
        input_device_index=self._device_index,
    )
    try:
        ...
```

The transcriber opens a fresh PyAudio stream per command. That matches the command-turn shape of the voice pipeline and keeps the wake-word stream and STT stream separate.

The loop has three useful exits:

1. Vosk detects end of utterance and returns a non-empty result.

2. Silence persists long enough that there is no partial speech, so the function returns an empty string.
3. The maximum recording window is reached and a final result is forced.

Observations

- The object is still deliberately small; the runtime now owns the higher-level command turn flow.
- A future improvement would be to make the silence timeout and record window part of the shared runtime dataclass instead of fixed class attributes. — version: “1.2” generated: “2026-05-06” —

IntentParser: Pure Intent Parsing Logic

`behavior_manager.py` holds `IntentParser` — pure logic for parsing and validating intents with no ROS2 dependency. The name `BehaviorManager` was retired in F15/T01; the class is now `IntentParser` to reflect its single responsibility.

The ROS2 node layer (`BehaviorManagerNode`) handles subscriptions and callbacks; `IntentParser` handles only the JSON parsing semantics. Tests for intent parsing run without a ROS2 context.

The Intent Dataclass

```
@dataclass
class Intent:
    name: str
    source: str
    slots: dict
```

An intent has a name (what to do), a source (who asked), and slots (parameters). The slots dict is open-ended — different intents carry different slot keys. Validation of slot contents is left to the downstream handler.

Parsing with Strict Validation

`parse_intent` takes a raw JSON string — as it arrives over a ROS2 topic — and returns a typed `Intent` or raises `ValueError`:

```
def parse_intent(self, payload: str) → Intent:
    try:
        raw = json.loads(payload)
    except json.JSONDecodeError as exc:
        raise ValueError(f"Invalid intent JSON: {exc.msg}") from exc

    if not isinstance(raw, dict):
        raise ValueError("Intent payload must be a JSON object")
```

```

name = raw.get("name")
if not isinstance(name, str) or not name:
    raise ValueError("Intent field 'name' must be a non-empty string")

source = raw.get("source", "unknown")
slots = raw.get("slots", {})
if not isinstance(slots, dict):
    raise ValueError("Intent field 'slots' must be an object")

return Intent(name=name, source=source, slots=slots)

```

Each check raises with a descriptive message. The caller (the ROS2 node) can log the error and discard the message without crashing.

Announcement Helpers

Two helpers are re-exported from this module:

```

from control.announcement_contract import (
    make_announcement_msg,
    make_announcement_payload,
)

```

The actual implementation lives in `announcement_contract.py` to avoid a circular dependency with the ROS2 message type.

Observations

- **No slot validation.** `parse_intent` validates structure but not slot contents. An intent `{"name": "count_objects", "slots": {}}` (missing `object_type`) will parse successfully and fail later.
- **source default is “unknown”.** Could be required rather than silently defaulted.
-

Single method class. `IntentParser` has one public method. The class form is kept for future expansion (e.g., intent history, rate limiting).

version: “1.0” generated: “2026-05-06” —

VoiceRuntime

`control/voice/runtime.py` is the shared non-ROS voice runtime. It collects the parameters copied from `~/tune`, runs the wake-word and STT helpers, and

returns a structured turn result that the ROS node can consume.

The point of this module is not flexibility for its own sake. It is a single place where the tuned voice parameters are visible, where the production audio flow is implemented, and where Pi-only smoke testing can happen without ROS.

Tuned Parameters

```
TUNED_VOICE_PARAMETERS = {
    "capture_card": 0,
    "playback_card": 0,
    "stream_settings": {
        "wake_word": "alexa",
        "threshold": 0.3,
        "wake_hits": 1,
        "live_filter": True,
        "vosk_model": "small",
        "grammar": list(DEFAULT_GRAMMAR),
    },
    "sox_chain": {
        "highpass": 120,
        "lowpass": 4000,
    },
}
```

This block is the hand-editable handoff point from tune. The code can use it directly, or it can load a tune YAML file for comparison. The important thing is that the production parameters are obvious and easy to paste.

Runtime Shape

The runtime is a small orchestration object:

```
class VoiceRuntime:
    def next_turn(self, ok_fn: Callable[[], bool] = lambda: True, ...):
        ...
```

next_turn() owns the whole voice turn:

1. Open or reuse the capture stream.
2. Wait for wake detection.
3. Capture the command.
4. Return a VoiceTurn dataclass with transcript text and timing metadata.

That return value is intentionally plain. The ROS adapter only needs to know what was recognized and whether the turn was empty.

Audio Processing

The runtime embeds the audio helpers that were validated in tune:

```
class LiveBandpass:
    def process(self, samples: np.ndarray) → np.ndarray:
        ...

def read_mono_chunk(stream, n_samples: int = CHUNK, live_filter=None) → np.ndarray | None:
    ...
```

The capture path does three things before recognition:

1. Convert stereo arecord output to mono.
2. Remove DC offset from each chunk.
3. Apply optional live highpass/lowpass filtering.

That keeps the runtime close to the tuned Pi audio path without pulling the tune package into control.

Wake And Command Flow

open_arecord → wait_for_wake → capture_command → VoiceTurn

The wake stage returns score and timing data so the smoke test can report useful metrics. The command stage strips the wake word from the recognized text before returning the final transcript.

Smoke Test

The module exposes a simple entry point:

```
python3 -m control.voice.runtime --trials 5
```

That is meant for Pi hardware, not unit tests. It gives a quick read on wake hits, empty turns, and latency using the pasted tuning parameters.

Observations

- The module is doing orchestration plus signal processing. That is acceptable here because the whole point is to have one place for the production voice behavior.
- If the voice contract grows, the next useful split would be to separate the audio helpers from the turn orchestration. — version: “2.0” generated: “2026-05-06” —

BehaviorManagerNode

behavior_manager_node.py is the ROS2 shell around IntentParser and the domain behavior handlers. It wires the behavior logic to ROS2 topics and services.

Node Topology

```
/intent (std_msgs/String)
  ↓ on_intent → IntentParser.parse_intent
BehaviorManagerNode
├── MotionBehavior.execute(intent, node)
│   ↓ RobotController (stop, drive_square, explore)
└── PerceptionBehavior.execute(intent, node)
    ↓ call_describe_scene (async service call)
    /describe_scene (std_srvs/Trigger)
    ↓ on_describe_scene_done
    /announcement (control/Announcement)
```

The node subscribes to `/intent`, routes the parsed intent to the first matching handler, and each handler drives its own downstream I/O.

Handler Registration

Handlers are registered as a list in `__init__`:

```
rc = RobotController(ConfigManager())
self.handlers = [
    MotionBehavior(rc),
    PerceptionBehavior(self),
]
```

`MotionBehavior` receives a `RobotController` instance. `PerceptionBehavior` receives the node itself so it can access the service client, publisher, and logger.

Intent Dispatch

```
def on_intent(self, msg: String) → None:
    try:
        intent = self.parser.parse_intent(msg.data)
    except ValueError as exc:
        self.get_logger().warn(str(exc))
        return

    for handler in self.handlers:
        if handler.handles(intent.name):
            handler.execute(intent, self)
            return

    self.get_logger().warn(f"Unhandled intent: {intent.name}")
```

The loop tries each handler in registration order. First match wins. Unrecognized intent names log a warning and are dropped — no crash, no retry.

Adding a New Domain

To add a new behavior domain:

1. Create `control/behaviors/my_domain_behavior.py` with `handles()` and `execute()`.
2. Append an instance to `self.handlers` in `BehaviorManagerNode.__init__`.
3. No changes to `on_intent` are needed.

Observations

- There is no `/intent` acknowledgement. Publishers have no way to know whether the intent was received and acted on.
- Handler order matters. The first handler whose `handles()` returns `True` claims the intent; later handlers never see it.
-

PerceptionBehavior receives the whole node. A narrower interface (publisher + client + logger) would reduce coupling.

version: "1.0" generated: "2026-05-06" —

VoiceInputNode

`voice_input_node.py` is now a ROS2 shell around the shared voice runtime. It no longer owns wake-word detection or speech recognition directly; instead it consumes a `VoiceTurn` from `control.voice.runtime`, maps the transcript to an intent, and publishes the ROS messages that the rest of the system expects.

Pipeline Shape

IDLE

```
→ runtime.next_turn()
→ LISTENING (beep at wake)
→ PROCESSING (map transcript)
→ SPEAKING (beep or "say again")
→ IDLE
```

The state topic remains a plain string topic because it is meant to be consumed by simple listeners such as a UI, a light indicator, or logs.

ROS Topics

Topic	Direction	Type	Purpose
/intent	publish	std_msgs/String	Mapped intent JSON
/voice/state	publish	std_msgs/String	IDLE / LISTENING / PROCESSING / SPEAKING
/announcement	publish	AnnouncementMsg	Failure feedback

The node only publishes an announcement on mapping failure. If the transcript is recognized and mapped, it publishes the intent and leaves any higher-level spoken response to the behavior manager or speech output node.

Wake Callback

The runtime accepts a small callback that fires when wake is detected. The ROS node uses that hook to publish LISTENING and play a short beep immediately.

```
def on_wake(_wake):
    node.get_logger().info("Wake word detected – speak your command")
    node.publish_state("LISTENING")
    beep(frequency=880, duration=0.02, device_index=device_index)
```

```
turn = runtime.next_turn(ok_fn=rclpy.ok, on_wake=on_wake)
```

That keeps the runtime free of ROS imports while preserving the user-facing timing of the listen state.

Transcript Handling

```
def process_turn(self, turn: VoiceTurn, device_index: int = 0) → None:
    if turn.empty:
        self.publish_state("SPEAKING")
        speak("say again")
        self.publish_announcement("I didn't catch that")
        return
    self.process_transcript(turn.text, device_index=device_index)
```

The node has one job after the runtime returns: turn the transcript into a normalized intent or a failure announcement. This is the layer where the voice contract meets the rest of the robot.

Observations

- The ROS loop still runs synchronously, so it does not process other callbacks while waiting on a voice turn.

- The `VOICE_DEVICE_INDEX` environment variable still selects the audio device. The value defaults to the runtime's capture card if the variable is not set.
- `SPEAKING` is still the published state name, although it mostly means “responding” now. — version: “1.0” generated: “2026-05-04” —

SpeechOutputNode

`speech_output_node.py` subscribes to `/announcement` and converts text to speech using Piper TTS and ALSA playback. It is the voice of the robot.

Pipeline

```
/announcement (AnnouncementMsg)
  ↓ on_announcement
  SpeechOutputNode
    ↓ synthesize_to_wav (Piper)
    /tmp/speech-output-XXXX.wav
      ↓ apply_wav_gain (optional)
    /tmp/speech-output-YYYY.wav
      ↓ play_wav (aplay)
    speaker output
      ↓ cleanup temp files
```

Piper TTS Integration

```
def synthesize_to_wav(text, wav_path, piper_bin, model_path, length_scale):
    cmd = [piper_bin, "--model", model_path, "--output_file", wav_path,
           "--length_scale", str(length_scale), "--quiet"]
    subprocess.run(cmd, input=text.encode("utf-8"), check=True, capture_output=True)
```

Piper is invoked as an external subprocess. Text is passed on stdin; the WAV file is written to a temp path. `--quiet` suppresses Piper's progress output. `length_scale` controls speech speed (`>1.0` = slower, `<1.0` = faster); the default is `1.25` — slightly slower than natural speed for clarity in a robot environment.

Volume Control via PCM Gain

Piper has no built-in volume control. Instead, the node applies a software gain to the raw PCM samples:

```
def _scale_pcm_frames(frames: bytes, sample_width: int, gain: float) → bytes:
    max_value = (1 << (sample_width * 8 - 1)) - 1
    min_value = -(1 << (sample_width * 8 - 1))
    for offset in range(0, len(frames), sample_width):
        sample = struct.unpack_from(fmt, frames, offset)[0]
```



```

        adjusted = max(min(int(sample * gain), max_value), min_value)
        scaled.extend(struct.pack(fmt, adjusted))
    return bytes(scaled)

```

The default `SPEECH_GAIN = 0.35` attenuates the output to 35% of full scale. This is a workaround for Piper models that generate audio at a high peak amplitude that would clip on the robot's speaker. Gain is applied in pure Python with `struct.pack/unpack` — not fast, but audio synthesis time (seconds) dominates over gain application time (milliseconds).

Environment Variable Configuration

All tunable parameters come from environment variables with sensible defaults:

Variable	Default	Purpose
PIPER_BIN	"piper"	Path to Piper binary
PIPER_MODEL_PATH	""	Required — raises <code>RuntimeError</code> if empty
SPEECH_ALSA_DEVICE	""	ALSA device string (e.g., "hw:1,0")
SPEECH_TMP_DIR	<code>tempfile.gettempdir()</code>	Directory for temp WAV files
SPEECH_GAIN	"0.35"	PCM amplitude multiplier
PIPER_LENGTH_SCALE	"1.25"	Speech rate (>1 = slower)

`PIPER_MODEL_PATH` is the only genuinely required variable — `synthesize_to_wav` raises `RuntimeError` at call time if it's empty.

Temp File Cleanup

```

def speak_text(self, text: str) → None:
    wav_path = self._make_wav_path()
    gained_path = None
    try:
        synthesize_to_wav(...)
        if self.speech_gain != 1.0:
            gained_path = self._make_wav_path()
            apply_wav_gain(wav_path, gained_path, self.speech_gain)
            playback_path = gained_path
        play_wav(playback_path, ...)
    finally:
        for path in (wav_path, gained_path):
            if path:

```

```

try: os.remove(path)
except FileNotFoundError: pass

```

Both temp files (original and gained) are cleaned up in finally. The `FileNotFoundError` suppression handles the case where synthesis failed before the file was created.

Observations

- `speak_text` blocks until Piper finishes synthesis and `aplay` finishes playback. During this time, /announcement messages queue in the ROS2 subscriber buffer (depth=10). If multiple announcements arrive while the robot is speaking, they will be spoken sequentially. There is no skip, interrupt, or priority-based preemption.
- `_scale_pcm_frames` only supports sample widths 1, 2, and 4 bytes. Piper always outputs 16-bit (2-byte) WAV, so this is not a practical limitation.
-

`parse_speech_gain` and `parse_length_scale` raise `ValueError` with descriptive messages. These are called in `__init__`, so a bad env var kills the node at startup with a clear error — good fail-fast behaviour.

version: “1.0” generated: “2026-05-06” —

MotionBehavior

`control/behaviors/motion_behavior.py` handles motion-domain intents. It has no ROS2 dependency — all execution goes through `RobotController`.

Interface

```
MOTION_INTENTS = {"stop", "explore", "drive_square"}
```

```

class MotionBehavior:
    def __init__(self, robot_controller): ...
    def handles(self, intent_name: str) → bool: ...
    def execute(self, intent: Intent, node=None) → None: ...

```

`handles` returns True for any name in `MOTION_INTENTS`. `execute` dispatches on `intent.name`:

```

def execute(self, intent: Intent, node=None) → None:
    if intent.name == "stop":
        self.rc.stop_robot()

```

```

elif intent.name == "drive_square":
    meters = float(intent.slots.get("meters", 1.0))
    self.rc.script_square(meters)
elif intent.name == "explore":
    pass # not yet implemented

```

node is accepted but unused — the signature matches `PerceptionBehavior` so both handlers can be called uniformly by `BehaviorManagerNode`.

Slot Handling

`drive_square` reads `slots["meters"]` with a default of `1.0`. All other current motion intents are parameterless. Slot validation is left to the caller; a missing or non-numeric meters will raise at the `float()` call.

Observations

- `explore` is a no-op. The implementation slot exists so the intent routes correctly without an “Unhandled intent” warning.
-

Adding a new motion intent requires: adding the name to `MOTION_INTENTS`, adding an `elif` branch in `execute`, and adding the intent name to `BEHAVIOR_COMMANDS` in `command_dispatcher.py` if CLI access is wanted.

version: “1.0” generated: “2026-05-06” —

PerceptionBehavior

`control/behaviors/perception_behavior.py` handles perception-domain intents via async ROS2 service calls. It holds no direct ROS2 imports at module load — `std_srvs.srv.Trigger` is imported lazily inside `call_describe_scene` to keep the module importable in no-ROS test environments.

Interface

```
PERCEPTION_INTENTS = {"describe_scene", "count_objects"}
```

```

class PerceptionBehavior:
    def __init__(self, node): ...
    def handles(self, intent_name: str) → bool: ...
    def execute(self, intent: Intent, node=None) → None: ...

```

PerceptionBehavior receives the owning BehaviorManagerNode at construction. It needs the node for three things: the describe_scene_client, the announcement_pub, and the logger.

Async Service Call Flow

```
execute("describe_scene")
    → call_describe_scene()
        → service_is_ready() check
        → lazy import std_srvs.srv.Trigger
        → call_async(Trigger.Request())
        → future.add_done_callback(on_describe_scene_done)
on_describe_scene_done(future)
    → future.result() → response
    → publish_announcement(response.message)
publish_announcement(text)
    → make_announcement_msg(text)
    → announcement_pub.publish(msg)
```

The call is non-blocking. `call_async` returns immediately; the callback fires when the ROS2 executor delivers the response. Blocking service calls inside spin callbacks would deadlock the executor.

Error Handling

`on_describe_scene_done` handles two failure modes:

1. **Exception from `future.result()`** (timeout, network error): logs error, returns.
2. **`response.success = False`**: logs warning, skips announcement.

In both cases the node continues running. There is no retry or queuing.

Observations

- `count_objects` logs “not yet implemented” and does nothing. The intent routes without warning.
- PerceptionBehavior receives the whole node. A narrower interface (client + publisher + logger) would reduce coupling.
-

`std_srvs.srv.Trigger` has no parameters. Passing slots data (e.g., `object_type`) to the scene service would require a richer service type.

version: “1.0” generated: “2026-05-06” —

IntentPublisher

control/commands/intent_publisher.py is the thin layer between CommandDispatcher and the /intent ROS2 topic. It accepts an injected publish_fn so unit tests stay No-ROS, while production uses IntentApi lazily.

Interface

```
class IntentPublisher:
    def __init__(self, publish_fn: Callable[[str], None] | None = None): ...
    def publish(self, name: str, source: str = "cli", slots: dict | None = None) → None: ...
    def get_api(self): ...
```

Publish Path

```
def publish(self, name: str, source: str = "cli", slots: dict | None = None) → None:
    payload = json.dumps({"name": name, "source": source, "slots": slots or {}})
    if self.publish_fn is not None:
        self.publish_fn(payload)
    else:
        self.get_api().publish(name, source, slots or {})
```

When publish_fn is injected, the JSON payload string is passed to it directly. This is the test path — callers capture the list and assert on parsed JSON. When publish_fn is None, get_api() creates an IntentApi instance on first call (lazy init avoids importing rclpy at module load).

Testing Pattern

```
published = []
pub = IntentPublisher(publish_fn=published.append)
dispatcher = CommandDispatcher(rc, intent_publisher=pub)
dispatcher.dispatch_text("scene describe")
assert json.loads(published[0])["name"] == "describe_scene"
```

The list collects raw JSON strings. Tests parse them to assert on structure.

CommandDispatcher Integration

CommandDispatcher.__init__ accepts intent_publisher=None. When None, publish_intent creates an IntentPublisher() inline (no injection, production default). When injected, the passed instance is used for all intent publishes.

Observations

- get_api() creates a new IntentApi(ConfigManager()). In a long-running process, the api attribute caches the instance after first use.

•

The split between **IntentPublisher** (JSON serialization + injection seam) and **IntentApi** (actual ROS2 publisher) exists so **IntentPublisher** remains importable without ROS2.

version: “1.0” generated: “2026-05-04” —

Parameter Definition (Appendix)

`parameter_def.py` defines a single dataclass describing one parameter a command can accept.

```
@dataclass
class ParameterDef:
    name: str          # kwarg name passed to RobotController method
    param_type: type    # e.g. float, bool, str – used for coercion
    required: bool      # if True, dispatcher raises ValueError when absent
    default: object     # None means "no default" (not the same as required=False)
    description: str    # shown in help text
```

`param_type` is the type itself (e.g. float), not a string, so `CommandDispatcher` can call `param_type(value)` directly. `bool` needs a special branch because `bool("false") == True`.

No business logic. No imports beyond `stdlib`. Everything else depends on this.

version: “1.0” generated: “2026-05-04” —

Command Definition (Appendix)

`command_def.py` defines a single dataclass describing one entry in the command registry.

```
@dataclass
class CommandDef:
    method_name: str          # method to call on RobotController
    parameters: list[ParameterDef] # ordered – position maps to CLI arg position
    description: str          # shown in help text
    group: str                # e.g. "movement", "launch", "config"
```

`method_name` decouples the CLI command name from the Python method name. The ordering of parameters is load-bearing: `SimpleCLI._map_to_dispatcher_format` maps CLI positional args to parameter names by index.

No business logic. No imports beyond `parameter_def`.

version: “1.0” generated: “2026-05-04” —

Control Commands (Appendix)

`control_commands.py` defines two basic robot control commands.

```
"robot.stop": CommandDef("stop_robot", [], "Stop robot movement", group="control")
"robot.status": CommandDef("get_robot_status", [], "Get current robot status", group="control")
```

Neither command takes parameters. `robot.stop` publishes a zero-velocity `Twist` via `MovementApi`. `robot.status` returns a structured status dict including speed config, tracked launch process PIDs, and which API nodes are initialised.

version: “1.0” generated: “2026-05-04” —

Navigation Commands (Appendix)

`navigation_commands.py` defines three map management commands.

```
"map.save": CommandDef("map_save", [], "Save current map to maps/ folder", group="map")
"map.list": CommandDef("list_maps", [], "List available maps in maps/ folder", group="map")
"map.serialize": CommandDef("map_serialize", [], "Save current map in SLAM Toolbox serialized form")
```

All three are parameterless — they read `map_name` from the config variable rather than from a CLI argument. `map.save` calls `nav2_map_server map_saver_cli`; `map.serialize` calls the SLAM Toolbox `serialize` service. Both are implemented in `RobotController` using `ProcessApi.run_command_sync`.

version: “2.0” generated: “2026-05-06” status: “removed” —

Intent Commands (Removed in F15/T07)

`intent_commands.py` defined CLI commands that published to `/intent` via `RobotController` methods (`publish_intent_stop`, `publish_intent_explore`, etc.). These were entries in the `CommandDispatcher` registry.

The file was deleted in F15/T07. Intent routing now lives in `BEHAVIOR_COMMANDS` in `command_dispatcher.py`, handled by `dispatch_text` without going through the registry. `IntentPublisher` replaced the `RobotController` intent methods.

See `12-command_dispatcher.md` for the current routing mechanism.

version: “2.0” generated: “2026-05-06” status: “removed” —

Semantic Commands (Removed in F15/T07)

`semantic_commands.py` defined user-friendly aliases (`scene.describe`, `scene.count`) that mapped to the same `RobotController` methods as the `intent.*` commands. These were entries in the `CommandDispatcher` registry.

The file was deleted in F15/T07. The `scene.*` aliases survive as entries in `BEHAVIOR_COMMANDS` in `command_dispatcher.py`, routed by `dispatch_text`.

See `12-command_dispatcher.md` for the current routing mechanism.

version: “1.0” generated: “2026-05-04” —

Audio Feedback (Appendix)

`audio_feedback.py` provides two lightweight audio feedback functions for voice state transitions.

beep

```
def beep(frequency: int = 880, duration: float = 0.15, device_index: int = 0) → None:
    sample_rate = 16000
    t = np.arange(int(sample_rate * duration)) / sample_rate
    samples = (np.sin(2 * math.pi * frequency * t) * 32767 * 0.05).astype(np.int16)
    # open PyAudio stream, write samples, close
```

Generates a pure sine wave at the given frequency. Amplitude is 5% of full scale (`* 0.05`) — intentionally quiet so it doesn’t overwhelm speech. Used at 880 Hz (“listening”) and 330 Hz (“intent received”) in `VoiceInputNode`.

speak

```
def speak(text: str, speed: int = 150, amplitude: int = 15) → None:
    subprocess.run(["espeak-ng", "-s", str(speed), "-a", str(amplitude), text], ...)
```


Calls **espeak-ng** for simple TTS. Used only for the “say again” fallback — the primary TTS pipeline uses Piper. Silent if **espeak-ng** is not installed (**FileNotFoundError** is caught and ignored).

version: “1.0” generated: “2026-05-04” —

Describe Scene Stub Node (Appendix)

`describe_scene_stub_node.py` is a temporary placeholder for the real `/describe_scene` service.

```
class DescribeSceneStubNode(Node):
    def __init__(self):
        super().__init__("describe_scene_stub")
        self.service = self.create_service(Trigger, "/describe_scene", self.describe_scene)

    def describe_scene(self, request, response):
        response.success = True
        response.message = "I see 2 cups and 1 can"
        return response
```

Returns a hardcoded string. Purpose: allows the behavior manager pipeline (BehaviorManagerNode → `/describe_scene` → `/announcement` → SpeechOutputNode) to be smoke-tested without the actual object detection stack running.

This node should be replaced with a real `/describe_scene` service backed by `/targets/confirmed` from the `oak__roboflow` detection pipeline. See `current.md` next steps.

version: “2.0” generated: “2026-05-06” status: “removed” —

SimpleCommandParser (Removed in F15/T05)

`simple_parser.py` was the text-to-command parser for the CLI. It converted free-form strings into `ParsedCommand` objects. The file was deleted in F15/T05.

What Replaced It

`CommandDispatcher.dispatch_text(text)` absorbed all functionality:

- Abbreviation resolution (`ABBREV_TO_FULL`, `FULL_NAMES`, `resolve_keyword`)
- Subcommand detection (checks `FULL_NAMES`, abbreviation expansion, and command registry)
- Value type inference (`parse_value`: `bool` → `int` → `float` → `str`)

`SimpleCLI.execute_command` now calls `dispatcher.dispatch_text(input_text)` directly. The intermediate `_map_to_dispatcher_format` step was also removed. See `12-command_dispatcher.md` for the current implementation.